PRODUCT REQUIREMENTS DOCUMENT

# Debug Relay Competition Platform

## Backend Engineering Specification

*For use as full context input to AI coding agents*

| | |
|---|---|
| **Runtime** | Node.js 20 LTS |
| **Database** | Supabase (PostgreSQL) |
| **Cache / Queue** | Redis + BullMQ |
| **WebSockets** | Socket.IO |
| **Code Execution** | OneCompiler API |
| **Auth** | Supabase Auth (JWT) |

# 1. Project Overview

Debug Relay is a university programming competition platform designed for real-time, relay-style debugging contests. The system supports 2 rounds of 30 or so minutes each, with a different team member competing in each round. Each round contains multiple debugging questions presented sequentially (approximately 2-3 minutes per question with a few seconds gap between them).

This document is the complete backend specification. It is written as a context document for AI coding agents and contains every architectural, data, logic, and concurrency decision made during design. An agent with this document as context should be able to implement the full backend without needing to ask clarifying questions about architecture.

## 1.1 Competition Format

- 2 rounds, 30 or more minutes each
- Each round is attempted by a different member of the same team (relay format — same member cannot play two rounds)
- Approximately 2-3 minutes per question, with a few seconds gap between questions
- Questions are pushed by the server automatically on a timer — participants do not manually advance questions
- Approximately 30 participants (teams) competing simultaneously

## 1.2 Run vs Submit Model

The platform distinguishes between Run and Submit. This is a core product decision:

- Run: Compiles the code and checks it against all test cases. Returns a detailed pass/fail breakdown. Does NOT record a score. Can be used unlimited times.
- Submit: Records the submission officially and triggers scoring. The Submit button is disabled in the UI until the current session has a passing Run result for that question. The server also enforces this — it rejects a Submit if no passing Run has been recorded for that team/question/round combination in Redis.
- Both Run and Submit go through the same BullMQ compilation pipeline and the same OneCompiler API call. The difference is only in what happens after a successful result: Run sets a Redis flag and returns test case details; Submit calculates score and updates the leaderboard.
- Submit always recompiles the code fresh — it does not reuse the Run result. This prevents scoring stale output if code was edited between Run and Submit.

# 2. System Architecture

The system is structured into six layers. All layers run within a single Node.js process (no microservices needed for this scale). Redis and PostgreSQL are external managed services.

## 2.1 Layer Overview (Top to Bottom)

| Layer | Technology | Responsibility |
|---|---|---|
| Client | React / Next.js | Participant code editor, admin dashboard, leaderboard display |
| API Gateway | Express.js + Socket.IO | REST endpoints, WebSocket server, JWT auth middleware |
| Core Services | Node.js modules | Competition engine, submission service, scoring engine, leaderboard service |
| Compilation | BullMQ + OneCompiler | Async job queue, code execution, test case validation |
| Cache / State | Redis (Upstash) | Live leaderboard, round state, dedup locks, run-pass flags, job storage |
| Persistence | Supabase PostgreSQL | All permanent data: teams, submissions, scores, questions, rounds |

## 2.2 Communication Protocols

- REST (HTTPS): Used for all request-response interactions — auth, submission intake, admin controls, fetching questions.
- WebSocket (Socket.IO): Used for all server-to-client push events — round start/end, question advancement, submission results, leaderboard updates. Participants connect once on page load and maintain this connection throughout the competition.
- Socket.IO rooms: All participants and the admin are joined to a single room named comp:{competitionId}. Broadcasts to this room hit all connected clients. Individual results (submission verdicts) are emitted to a specific socket ID, not broadcast to the room.

## 2.3 WebSocket Event Catalog

| Event Name | Direction | Payload / Purpose |
|---|---|---|
| competition:state | Server → Client | Sent on connection. Full snapshot: round status, current question index, time remaining, leaderboard. |
| round:start | Server → All | Round number, first question payload, round duration in seconds. |

| | | |
|---|---|---|
| question:next | Server → All | Next question payload. Fired by server-side timer automatically. |
| round:end | Server → All | Round summary scores. Triggers relay handoff prompt on client. |
| submission:result | Server → One | Verdict (pass/fail), total score earned, bonus score, test case breakdown. Sent to submitting team only. |
| run:result | Server → One | Test case pass/fail details for Run. Does not include score. Sent to team only. |
| leaderboard:update | Server → All | Updated top-N rankings. Throttled to max 1 broadcast per second. |

# 3. Database Schema (PostgreSQL / Supabase)

All tables live in Supabase (PostgreSQL). Row-level security (RLS) is enabled. Teams can only read/write their own rows. Admins bypass RLS.

## 3.1 Table Definitions

### teams

| Column | Definition |
|---|---|
| id | UUID PRIMARY KEY DEFAULT gen_random_uuid() |
| name | TEXT NOT NULL — display name of the team |
| code | TEXT UNIQUE NOT NULL — join code used at login (e.g. ALPHA42) |
| created_at | TIMESTAMPTZ DEFAULT now() |

### users

| Column | Definition |
|---|---|
| id | UUID PRIMARY KEY DEFAULT gen_random_uuid() |
| team_id | UUID REFERENCES teams(id) NOT NULL |
| name | TEXT NOT NULL — member name provided at login |
| round_number | INTEGER NOT NULL — which round this member is assigned to (1, 2, or 3) |
| created_at | TIMESTAMPTZ DEFAULT now() |
| UNIQUE | UNIQUE(team_id, round_number) — enforces one member per round per team |

### rounds

| Column | Definition |
|---|---|
| id | UUID PRIMARY KEY DEFAULT gen_random_uuid() |
| round_number | INTEGER NOT NULL — 1, 2, or 3 |
| status | TEXT NOT NULL DEFAULT 'IDLE' — one of: IDLE, ACTIVE, PAUSED, ENDED |
| started_at | TIMESTAMPTZ — set when admin triggers round start |
| ended_at | TIMESTAMPTZ — set when 30-min timer fires or admin ends round |

| | |
|---|---|
| duration_seconds | INTEGER NOT NULL DEFAULT 1800 — configurable round length |

## questions

| Column | Definition |
|---|---|
| id | UUID PRIMARY KEY DEFAULT gen_random_uuid() |
| round_id | UUID REFERENCES rounds(id) NOT NULL |
| position | INTEGER NOT NULL — order within the round (0-indexed) |
| title | TEXT NOT NULL |
| description | TEXT NOT NULL — full problem statement with buggy code |
| test_cases | JSONB NOT NULL — array of {input: string, expected_output: string} |
| time_limit_seconds | INTEGER NOT NULL DEFAULT 150 — window for this question (2.5 min default) |
| base_score | INTEGER NOT NULL DEFAULT 100 |
| UNIQUE | UNIQUE(round_id, position) |

**Note:** test_cases is a JSONB array. Each element has two fields: input (stdin string) and expected_output (expected stdout string). The validator trims and normalizes whitespace before comparing.

## submissions

| Column | Definition |
|---|---|
| id | UUID PRIMARY KEY DEFAULT gen_random_uuid() |
| user_id | UUID REFERENCES users(id) NOT NULL |
| team_id | UUID REFERENCES teams(id) NOT NULL |
| question_id | UUID REFERENCES questions(id) NOT NULL |
| round_id | UUID REFERENCES rounds(id) NOT NULL |
| job_type | TEXT NOT NULL — 'run' or 'submit' |
| code | TEXT NOT NULL |
| language | TEXT NOT NULL — e.g. 'python3', 'java', 'cpp' |
| status | TEXT NOT NULL DEFAULT 'PENDING' — PENDING, COMPILING, ACCEPTED, WRONG_ANSWER, ERROR, TIMEOUT |
| submitted_at | TIMESTAMPTZ NOT NULL DEFAULT now() — ALWAYS set server-side, never from client |
| result | JSONB — {passed: number, total: number, test_results: [{input, expected, actual, passed}]} |

| | |
|---|---|
| base_score | INTEGER — null until evaluated |
| bonus_score | INTEGER — null until evaluated |
| total_score | INTEGER GENERATED ALWAYS AS (COALESCE(base_score,0) + COALESCE(bonus_score,0)) STORED |
| solve_rank | INTEGER — 1 = first team to correctly solve this question |

### leaderboard_snapshots

| Column | Definition |
|---|---|
| id | UUID PRIMARY KEY DEFAULT gen_random_uuid() |
| competition_id | UUID NOT NULL |
| round_id | UUID REFERENCES rounds(id) NOT NULL |
| snapshot | JSONB NOT NULL — full leaderboard array at time of snapshot |
| created_at | TIMESTAMPTZ DEFAULT now() |

**Important:** leaderboard_snapshots is written ONCE per round — when the round ends (status transitions to ENDED). It is never written during live operation. It serves as a historical record for post-competition review, replay, and dispute resolution. The live leaderboard during competition is served entirely from Redis.

The snapshot JSONB has this structure:

```
{ "captured_at": "ISO8601", "rankings": [ { "rank": 1, "team_id": "uuid",
"team_name": "Alpha", "total_score": 510, "round_scores": { "1": 510, "2": 0,
"3": 0 }, "questions_solved": 4 }, ... ] }
```

# 4. Redis — Key Reference

Redis is used as the live working memory of the competition. It is NOT a replacement for PostgreSQL — important data is always persisted to Postgres. Redis holds state that needs to be read/written at high frequency or needs atomic operations. Use Upstash (serverless Redis) for easy deployment.

## 4.1 Key Definitions

| Redis Key Pattern | Type | Purpose & Behaviour |
|---|---|---|
| round:{roundId}:status | STRING | Current round state. Values: IDLE, ACTIVE, PAUSED, ENDED. Written on every state transition. Read on every WebSocket reconnect and submission validation. |
| round:{roundId}:start_at | STRING | Unix timestamp (ms) when round became ACTIVE. Used to calculate time elapsed and time remaining. |
| round:{roundId}:current_question | STRING | 0-indexed position of the current question being displayed. Incremented by server timer. |
| leaderboard:{compId} | SORTED SET | Keys are team IDs, scores are total competition scores. ZADD on each accepted submit. ZREVRANGE to get rankings. This is the live source of truth for the leaderboard. |
| dedup:{teamId}:{questionId} | STRING (NX) | Duplicate submission lock. SET NX with EX 60 (60 second TTL). If key exists, reject submit with 429. Atomic operation — only one concurrent submit can win. |
| runpass:{teamId}:{questionId}:{roundId} | STRING | Set to '1' when a Run returns all test cases passed for this team/question/round. Checked on Submit — if absent, return 403 Forbidden. TTL: expires at round end. |
| comp:{compId}:state | HASH | Full competition state snapshot: active round, question index, status. Used for server restart recovery — server rehydrates from this on boot. |

## 4.2 Leaderboard Sorted Set Operations

The leaderboard is maintained as a Redis Sorted Set. All commands are O(log N) or O(N log N):

```
// Add or update a team score (called after each accepted submission):
ZADD leaderboard:{compId} {newTotalScore} {teamId}
```

```
// Get top 30 teams in ranked order (for broadcast):
ZREVRANGE leaderboard:{compId} 0 29 WITHSCORES

// Get a specific team's rank (0-indexed from top):
ZREVRANK leaderboard:{compId} {teamId}
```

## 4.3 Dedup Lock Pattern

This prevents double-scoring from network retries or double-clicks. The SET NX operation is atomic in Redis — only one caller can set a key that does not exist:

```
// On submit request arrival:
const result = await redis.set(`dedup:${teamId}:${questionId}`, 1, 'NX', 'EX',
60);
if (result === null) {
  return res.status(429).json({ error: 'Duplicate submission' });
}
// If result is 'OK', lock was acquired — proceed with submission
```

# 5. BullMQ — Compilation Queue

BullMQ is a Node.js job queue library backed by Redis. It manages the asynchronous pipeline between receiving a code submission and calling the OneCompiler API. This decoupling is essential: without a queue, 30 simultaneous submissions would flood the OneCompiler API and block the Express server waiting for HTTP responses.

## 5.1 Queue Configuration

```
import { Queue, Worker } from 'bullmq';
import { redis } from './redis.js'; // shared ioredis connection

const compileQueue = new Queue('compile', { connection: redis });

const worker = new Worker('compile', processCompileJob, {
  connection: redis,
  concurrency: 5,  // max 5 OneCompiler API calls simultaneously
  limiter: { max: 10, duration: 1000 } // max 10 jobs per second
});
```

## 5.2 Job Payload

When a Run or Submit is received, a job is enqueued with this payload:

```
await compileQueue.add('compile', {
  submissionId: 'uuid',
  teamId: 'uuid',
  questionId: 'uuid',
  roundId: 'uuid',
  userId: 'uuid',
  socketId: 'socket.io-id', // for targeted result delivery
  code: '...user code...',
  language: 'python3',
  testCases: [{ input: '5', expected_output: '25' }],
  jobType: 'run' | 'submit',  // determines post-processing
  questionBaseScore: 100,
  submittedAt: Date.now(), // server-recorded timestamp
}, { attempts: 2, backoff: { type: 'fixed', delay: 2000 } });
```

## 5.3 Worker Logic (processCompileJob)

The worker function handles both run and submit job types:

1. Update submission status to COMPILING in PostgreSQL.

2. POST to OneCompiler API with { language, stdin, files: [{ name, content: code }] }. Set a 10-second axios timeout.
3. On OneCompiler response, compare actual stdout against expected_output for each test case (trim whitespace, normalize line endings).
4. Build result object: { passed, total, test_results: [{input, expected, actual, passed}] }.
5. If jobType === 'run': Set runpass:{teamId}:{questionId}:{roundId} = 1 in Redis if all passed. Emit run:result via Socket.IO to socketId. Update submission record in Postgres. STOP — no scoring.
6. If jobType === 'submit' and all tests passed: Call Scoring Engine (section 6). Emit submission:result to socketId. Broadcast leaderboard:update to all.
7. If jobType === 'submit' and tests failed: Update submission status to WRONG_ANSWER in Postgres. Emit submission:result with failure details to socketId.
8. On OneCompiler timeout or 5xx: Mark submission as ERROR. If attempts remaining, BullMQ retries automatically after 2 seconds.

# 6. Scoring Engine

## 6.1 Scoring Model — Rank-Based Bonus (Recommended)

The scoring model uses a base score plus a rank-based bonus. This is preferred over time-decay because it is fair across different network latencies, easy to explain to participants, and immune to clock skew.

```
totalScore = baseScore + bonusScore

bonusScore = RANK_BONUSES[solveRank - 1]  // 0-indexed
RANK_BONUSES = [50, 40, 30, 20, 10, 5, 0, 0, 0, ...]
// 1st correct solver: +50, 2nd: +40, 3rd: +30, 4th: +20, 5th: +10, 6th+: +5
```

baseScore is defined per question in the questions table (default 100). solveRank is determined atomically using a PostgreSQL transaction with SELECT FOR UPDATE to prevent two simultaneous correct solvers getting the same rank.

## 6.2 Solve Rank Determination (Concurrency-Safe)

This is the critical section. Two teams may submit a correct answer within milliseconds of each other. The following PostgreSQL transaction ensures each team gets a unique rank:

```
BEGIN;
-- Count how many accepted submissions exist for this question:
SELECT COUNT(*) FROM submissions
  WHERE question_id = $questionId AND status = 'ACCEPTED'
  FOR UPDATE; -- row-level lock
-- solveRank = COUNT + 1
UPDATE submissions SET
  status = 'ACCEPTED',
  solve_rank = {solveRank},
  base_score = {baseScore},
  bonus_score = {bonusScore}
WHERE id = $submissionId;
COMMIT;
```

## 6.3 Team Total Score

A team's leaderboard score is the sum of all accepted submission scores across all rounds. After updating the submission in Postgres, the scoring engine queries the team's new total:

```
SELECT COALESCE(SUM(total_score), 0) AS team_total
FROM submissions
WHERE team_id = $teamId AND status = 'ACCEPTED';
```

```
// Then update Redis leaderboard:
await redis.zadd(`leaderboard:${compId}`, teamTotal, teamId);
```

## 6.4 Retry Penalty

Each failed Submit attempt (WRONG_ANSWER) deducts 5 points from the base score on the eventual accepted submission, down to a floor of 10 points. The deduction is applied at score-calculation time by counting prior WRONG_ANSWER submissions for the same team/question:

```
const wrongAttempts = await countWrongAnswers(teamId, questionId);
const effectiveBaseScore = Math.max(10, baseScore - (wrongAttempts * 5));
```

**Note:** Run attempts (jobType = 'run') do NOT count toward the retry penalty. Only Submit attempts with WRONG_ANSWER status count. This encourages using Run freely.

# 7. Competition Engine

The Competition Engine is a Node.js module (not a separate process) that manages round state transitions and the server-side question advancement timer. All state is persisted to Redis so the engine can recover if the server restarts.

## 7.1 Round State Machine

| State | Description |
|---|---|
| IDLE | Default state. No round active. Submissions rejected. |
| ACTIVE | Round in progress. Questions advancing on timer. Submissions accepted. |
| PAUSED | Admin has paused the round. Timer paused. Submissions rejected. Used for emergencies. |
| ENDED | 30-minute timer expired or admin ended round. Submissions rejected. Leaderboard snapshot written. |

## 7.2 Round Start Sequence

9. Admin calls POST /admin/round/:roundId/start
10. Engine validates: round exists, status is IDLE, admin is authenticated
11. Engine writes to Redis: round:{roundId}:status = ACTIVE, round:{roundId}:start_at = Date.now(), round:{roundId}:current_question = 0
12. Engine updates rounds table in PostgreSQL: status = ACTIVE, started_at = now()
13. Engine fetches question[0] from PostgreSQL
14. Socket.IO broadcasts round:start to entire competition room with question[0] payload and round duration
15. Engine schedules question advancement timer using setInterval anchored to start_at (not a raw setTimeout, so drift-free)

## 7.3 Question Advancement

Questions advance automatically on a server-side timer. The timer is not trusting the elapsed interval alone — it calculates actual elapsed time from the stored start_at to handle server hiccups:

```
// On each timer tick:
const elapsed = Date.now() - roundStartAt;
const expectedQuestion = Math.floor(elapsed / questionWindowMs);
const currentQuestion = await redis.get(`round:${roundId}:current_question`);
if (expectedQuestion > currentQuestion && expectedQuestion < totalQuestions) {
  await redis.set(`round:${roundId}:current_question`, expectedQuestion);
  const question = questions[expectedQuestion];
  io.to(`comp:${compId}`).emit('question:next', { question, index:
expectedQuestion });
```

```
}
```

## 7.4 Round End Sequence

16. 30-minute timer fires (or admin calls POST /admin/round/:roundId/end)
17. Engine sets round:{roundId}:status = ENDED in Redis
18. Engine updates rounds table: status = ENDED, ended_at = now()
19. Engine queries full leaderboard from Redis ZREVRANGE
20. Engine enriches leaderboard with team names and per-round scores from PostgreSQL
21. Engine writes one row to leaderboard_snapshots with the enriched JSON
22. Socket.IO broadcasts round:end to all clients with round summary
23. All runpass:{teamId}:*:{roundId} keys are deleted from Redis (cleanup)

# 8. REST API Endpoints

## 8.1 Auth Endpoints

| Method + Path | Auth Required | Description |
|---|---|---|
| POST /auth/login | None | Body: { team_code, member_name, round_number }. Validates team_code against teams table. Creates or retrieves user record. Enforces UNIQUE(team_id, round_number). Returns Supabase JWT. |
| POST /auth/logout | JWT | Invalidates session. Disconnects associated WebSocket. |

## 8.2 Participant Endpoints

| Method + Path | Auth Required | Description |
|---|---|---|
| POST /run | JWT (participant) | Body: { code, language, questionId, roundId }. Validates round ACTIVE, question is current question. Enqueues BullMQ job with jobType=run. Returns 202 immediately. |
| POST /submit | JWT (participant) | Body: { code, language, questionId, roundId }. Validates round ACTIVE, checks runpass Redis key (403 if absent), runs dedup lock (429 if exists), inserts submission, enqueues BullMQ job with jobType=submit. Returns 202 immediately. |
| GET /competition/:compId/state | JWT | Returns current round, question index, time remaining, and latest leaderboard from Redis. Used on reconnect. |

## 8.3 Admin Endpoints

All admin endpoints require JWT with admin role claim (set in Supabase Auth user metadata).

| Method + Path | Description |
|---|---|
| POST /admin/round/:roundId/start | Transitions round from IDLE to ACTIVE. Starts question timer. Broadcasts round:start. |
| POST /admin/round/:roundId/end | Force-ends a round before 30-min timer. Writes leaderboard snapshot. Broadcasts round:end. |
| POST /admin/round/:roundId/pause | Pauses the round. Freezes timer. Broadcasts round:paused event. |

| | |
|---|---|
| POST /admin/round/:roundId/resume | Resumes a paused round. Restores timer offset. Broadcasts round:resumed. |
| GET /admin/submissions | Paginated list of all submissions with status, scores, and team info. For monitoring. |
| PATCH /admin/submission/:id/score | Override a submission's score manually. Updates Postgres and Redis leaderboard. |

# 9. Concurrency & Edge Cases

This section documents every known concurrency challenge and the exact mechanism used to address it. Implementing agents must not introduce alternative approaches that bypass these protections.

| Challenge | Solution | Implementation Detail |
|---|---|---|
| Double-click / network retry duplicate submissions | Redis SET NX dedup lock | SET dedup:{teamId}:{questionId} 1 NX EX 60. If returns null, lock exists — reject 429. Atomic at Redis level. |
| 30 simultaneous submissions flooding OneCompiler | BullMQ concurrency limit | Worker concurrency: 5. All 30 jobs queue up, max 5 run at a time. FIFO processing. |
| Two teams solving simultaneously get same solve_rank | PostgreSQL SELECT FOR UPDATE | Transaction with FOR UPDATE on submissions count. Serializes rank assignment. |
| Server restart mid-round loses timer and state | Redis persistence for all live state | All round state stored in Redis. On boot, server reads comp:{compId}:state and resumes timers from stored start_at. |
| Leaderboard read bottleneck under high write load | Redis sorted set + async Postgres write | Leaderboard served from Redis only during competition. Postgres write happens asynchronously after Redis update. |
| WebSocket broadcast storm (30 clients × frequent updates) | Throttled broadcasts | leaderboard:update throttled to max 1 per second using lodash.throttle. Sends ranked diff, not full list. |
| Client-provided timestamps manipulated for scoring | Server-side timestamps only | submitted_at is set by server using Date.now() at moment of request receipt. Client timestamp is never used. |
| Same team member playing two rounds (relay violation) | DB UNIQUE constraint + middleware | UNIQUE(team_id, round_number) in users table. Enforced at login — system refuses to create duplicate assignment. |
| Submit before passing Run (bypassing disabled button) | Server-side runpass check | Server checks Redis runpass:{teamId}:{questionId}:{roundId} key. Returns 403 if absent, regardless of client UI state. |
| OneCompiler API timeout or 5xx error | BullMQ retry with backoff | Job configured with attempts: 2, backoff: fixed 2000ms. After 2 failures, submission marked ERROR and user notified via WS. |

# 10. Complete Request Flows

## 10.1 Auth & Session Setup

24. Browser POSTs /auth/login with { team_code, member_name, round_number }
25. Express validates team_code against teams table
26. Server creates user record (or retrieves if reconnecting). Enforces UNIQUE(team_id, round_number) — returns 409 if another member already has this round
27. Supabase Auth issues JWT embedding { user_id, team_id, round_number, role: 'participant' }
28. Browser connects Socket.IO with JWT in auth header
29. Server validates JWT on connection. Joins socket to room comp:{competitionId}
30. Server emits competition:state with round status, current question, time remaining, leaderboard snapshot

## 10.2 Run Flow

31. Browser POSTs /run with { code, language, questionId, roundId } + JWT
32. Middleware validates JWT. Checks round:{roundId}:status = ACTIVE in Redis. Checks questionId matches current_question in Redis.
33. Server inserts submission record to Postgres with job_type='run', status='PENDING', submitted_at=now()
34. Server enqueues BullMQ job with jobType='run' and all required fields. Returns 202 Accepted immediately.
35. BullMQ Worker picks up job. Updates submission status to COMPILING.
36. Worker POSTs to OneCompiler API. Awaits response (10s timeout).
37. Worker compares stdout for each test case against expected_output (trimmed, normalized).
38. If all pass: SET runpass:{teamId}:{questionId}:{roundId} = 1 EX {roundRemainingSeconds} in Redis.
39. Worker emits run:result to participant's socketId with full test case breakdown.
40. Worker updates submission record in Postgres with status=ACCEPTED and result JSON.

## 10.3 Submit Flow

41. Browser POSTs /submit with { code, language, questionId, roundId } + JWT
42. Middleware validates JWT. Checks round ACTIVE in Redis.
43. Server checks Redis for runpass:{teamId}:{questionId}:{roundId}. Returns 403 if absent.
44. Server executes Redis SET NX on dedup:{teamId}:{questionId}. Returns 429 if key exists.
45. Server inserts submission record with job_type='submit', status='PENDING', submitted_at=now()
46. Server enqueues BullMQ job with jobType='submit'. Returns 202 Accepted immediately.
47. Worker picks up job, calls OneCompiler (same as Run flow, steps 5-7 above).
48. If tests pass: Scoring Engine calculates solve_rank (Postgres transaction with FOR UPDATE). Calculates effectiveBaseScore (minus 5 per prior wrong attempt, floor 10). Calculates bonusScore from RANK_BONUSES array.
49. Scoring Engine: UPDATE submission with status=ACCEPTED, solve_rank, base_score, bonus_score.

50. Scoring Engine: Query team's new total from Postgres. ZADD leaderboard:{compId} {total} {teamId} in Redis.
51. Worker emits submission:result to participant's socketId with verdict and scores.
52. Worker emits leaderboard:update (throttled) to all clients in competition room.
53. If tests fail: submission updated to WRONG_ANSWER. run:result emitted to participant with failure details. dedup key is deleted so team can try again.

## 10.4 Round End Flow

54. 30-minute setInterval fires in Competition Engine (or admin calls /admin/round/:roundId/end)
55. Engine sets round:{roundId}:status = ENDED in Redis
56. Engine updates rounds table in Postgres: status=ENDED, ended_at=now()
57. Engine fetches ZREVRANGE leaderboard:{compId} 0 -1 WITHSCORES from Redis
58. Engine enriches data with team names, per-round scores from Postgres
59. Engine writes ONE row to leaderboard_snapshots with full snapshot JSON
60. Engine clears all runpass:*:*:{roundId} keys from Redis
61. Socket.IO broadcasts round:end with round summary to all clients

# 11. Technology Stack

| Concern | Technology | Rationale |
|---|---|---|
| Runtime | Node.js 20 LTS | Required. Event loop handles 30 concurrent WS connections trivially. Use cluster module if needed. |
| HTTP Framework | Express.js | Standard choice. Add helmet, cors, express-rate-limit middlewares from the start. |
| WebSockets | Socket.IO 4.x | Auto-fallback to long-polling, built-in rooms, reconnection handling, auth middleware hooks. |
| Database | Supabase (PostgreSQL) | Free tier sufficient for ~30 participants. Built-in Auth. RLS policies. Admin dashboard for live queries. |
| Cache / Queue backend | Redis via Upstash | Serverless Redis. Free tier sufficient. Supports BullMQ, sorted sets, atomic operations. |
| Job Queue | BullMQ | Redis-backed, supports concurrency limits, retries, backoff, job history. Standard for Node.js async processing. |
| Auth | Supabase Auth | Issues JWTs automatically. Use custom claims for team_id and round_number. Validate on server with Supabase admin client. |
| HTTP Client | Axios | For OneCompiler API calls. Supports timeout config and interceptors for retry logic. |
| ORM / DB Client | Supabase JS Client + pg (node-postgres) | Supabase client for standard queries. Raw pg for transactions with FOR UPDATE. |
| Deployment | Railway or Render | One-click Node.js deploy. Free tier handles 30 users. Alternatively: single VPS with PM2. |

## 11.1 NPM Dependencies

| Package | Version / Purpose |
|---|---|
| express | ^4.18 — HTTP server |
| socket.io | ^4.7 — WebSocket server |
| bullmq | ^5.x — Job queue |
| ioredis | ^5.x — Redis client (used by BullMQ and directly) |
| @supabase/supabase-js | ^2.x — Database and Auth client |

| | |
|---|---|
| pg | ^8.x — Raw PostgreSQL for transactions |
| axios | ^1.x — OneCompiler API HTTP calls |
| jsonwebtoken | ^9.x — JWT validation |
| express-rate-limit | ^7.x — Rate limiting on submission endpoints |
| helmet | ^7.x — Security headers |
| lodash.throttle | ^4.x — Leaderboard broadcast throttle |
| dotenv | ^16.x — Environment variable management |

# 12. Environment Variables

| Variable | Description |
|---|---|
| PORT | HTTP server port. Default 3000. |
| SUPABASE_URL | Supabase project URL from dashboard. |
| SUPABASE_SERVICE_KEY | Supabase service role key (bypasses RLS). Never expose to client. |
| SUPABASE_ANON_KEY | Supabase anon key for client-side auth only. |
| JWT_SECRET | Secret for verifying Supabase JWTs. Found in Supabase dashboard under API settings. |
| REDIS_URL | Upstash Redis URL. Format: redis://:{password}@{host}:{port} |
| ONECOMPILER_API_KEY | API key for OneCompiler. Required in Authorization header. |
| ONECOMPILER_API_URL | https://onecompiler-apis.p.rapidapi.com/api/v1/run |
| COMPETITION_ID | UUID of the active competition. Set once before event starts. |
| ADMIN_EMAILS | Comma-separated list of admin email addresses for role assignment. |
| QUESTION_WINDOW_SECONDS | Duration per question in seconds. Default 150 (2.5 min). |
| ROUND_DURATION_SECONDS | Duration per round in seconds. Default 1800 (30 min). |
| MAX_CONCURRENT_COMPILE_JOBS | BullMQ worker concurrency. Default 5. |
| LEADERBOARD_BROADCAST_INTERVAL_MS | Throttle interval for leaderboard WS broadcasts. Default 1000. |

# 13. Recommended Project Structure

```
src/
  index.js                  — Entry point: starts Express + Socket.IO + BullMQ
worker
  config/
    env.js                  — Validated environment variables
    redis.js                — ioredis singleton instance
    supabase.js             — Supabase admin client singleton
  routes/
    auth.js                 — POST /auth/login, /auth/logout
    participant.js          — POST /run, POST /submit, GET
/competition/:id/state
    admin.js                — All /admin/* routes
  middleware/
    auth.js                 — JWT validation middleware
    adminOnly.js            — Admin role check
    rateLimiter.js          — express-rate-limit configs
  services/
    competitionEngine.js    — State machine, round timers, question advancement
    submissionService.js    — Dedup lock, Postgres insert, queue enqueue
    scoringEngine.js        — Solve rank, score calculation, Redis + Postgres
update
    leaderboardService.js   — Redis sorted set operations, broadcast throttle
  workers/
    compileWorker.js        — BullMQ worker: OneCompiler call, test validation,
branching
  queues/
    compileQueue.js         — BullMQ Queue instance
  socket/
    index.js                — Socket.IO server setup, event handlers, room
management
  db/
    queries.js              — All SQL queries and Supabase JS calls
    transactions.js         — Raw pg transactions (solve rank)
```

# 14. Critical Implementation Notes for AI Agents

These are non-negotiable constraints. Do not deviate from these without explicit instruction.

- NEVER use client-provided timestamps for any scoring or ordering logic. Always use Date.now() on the server at the moment of request receipt.
- NEVER reuse a Run result for a Submit. Always recompile fresh on Submit. This prevents scoring stale output.
- NEVER write to leaderboard_snapshots during live competition. Only write once per round, at round end, during the round end sequence.

- ALWAYS set submitted_at using DEFAULT now() in Postgres (server-side) — never pass it from the client request body.
- The dedup Redis key must be deleted when a submission receives WRONG_ANSWER so the team can resubmit. Do not leave the lock in place on failure.
- The runpass Redis key TTL should be set to the remaining seconds in the current round, not a fixed value. This ensures keys expire naturally when the round ends.
- Socket.IO leaderboard broadcasts must be throttled to max 1 per second using lodash.throttle. Do not broadcast on every single score update.
- BullMQ worker concurrency must be configurable via MAX_CONCURRENT_COMPILE_JOBS environment variable. Default 5.
- All Redis operations that need to be atomic (dedup lock, leaderboard update) must use single Redis commands — not multiple commands in sequence. Use ZADD for leaderboard, SET NX for locks.
- Admin endpoints must check both JWT validity AND admin role claim. A valid participant JWT must not be able to access admin routes.
- The solve rank Postgres transaction must use SELECT FOR UPDATE to serialize concurrent correct submissions. Do not use application-level locking.

**For AI Agents:** This document is the complete specification. Do not invent features, alternative approaches, or additional tables not documented here. If something is ambiguous, implement the most conservative/safe interpretation and add a TODO comment.